
PRACTICAL WEEK 11

1. Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical11.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

long counter = 0;

void *increment_counter(void *arg)
{
    for (long i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Race Condition Demonstration\n");
    printf("Threads: %d\n", NUM_THREADS);
    printf("Iterations per thread: %d\n", ITERATIONS);

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL, increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("\nExpected counter value: %d\n",
        NUM_THREADS * ITERATIONS);
}
```

```

    printf("Actual counter value: %ld\n", counter);

    if (counter == NUM_THREADS * ITERATIONS)
        printf("Result: Correct\n");
    else
        printf("Result: Race condition detected\n");

    return 0;
}

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical11.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical11.1.c -o practical11.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical11.1

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical11.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical11.1.c -o practical11.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical11.1
Race Condition Demonstration
Threads: 4
Iterations per thread: 1000000

Expected counter value: 40000000
Actual counter value: 1062936
Result: Race condition detected
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

2. Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

Create a file:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical11.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

long counter = 0;

pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;

void *increment_counter(void *arg)
{
    for (long i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&counter_mutex);

        counter++;

        pthread_mutex_unlock(&counter_mutex);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Mutex Synchronization Demonstration\n");
    printf("Threads: %d\n", NUM_THREADS);
    printf("Iterations per thread: %d\n", ITERATIONS);

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL, increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {

```

```

        pthread_join(threads[i], NULL);
    }

    printf("\nExpected counter value: %d\n",
        NUM_THREADS * ITERATIONS);

    printf("Actual counter value: %ld\n", counter);

    if (counter == NUM_THREADS * ITERATIONS)
        printf("Result: Correct - synchronization successful\n");
    else
        printf("Result: Incorrect\n");

    pthread_mutex_destroy(&counter_mutex);

    return 0;
}

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical11.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical11.2.c -o practical11.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical11.2

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical11.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical11.2.c -o practical11.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical11.2
Mutex Synchronization Demonstration
Threads: 4
Iterations per thread: 1000000

Expected counter value: 4000000
Actual counter value: 4000000
Result: Correct - synchronization successful
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```